Fundamentos de Programación

Grado en Ingeniería de Computadores

8/10/26

Antecedentes

- Tuplas y tuplas con nombre
- Lectura
- Filtrado
- Transformación

Índice

- Paso de parámetros
- Type hinting

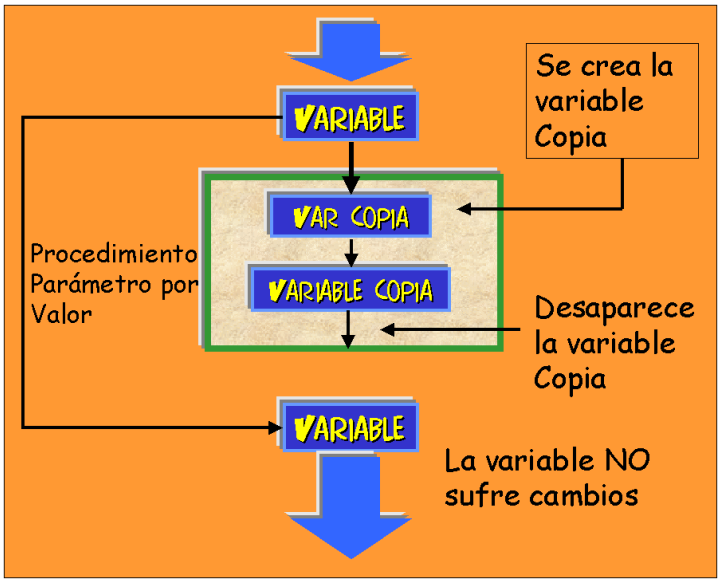
PASO DE PARÁMETROS

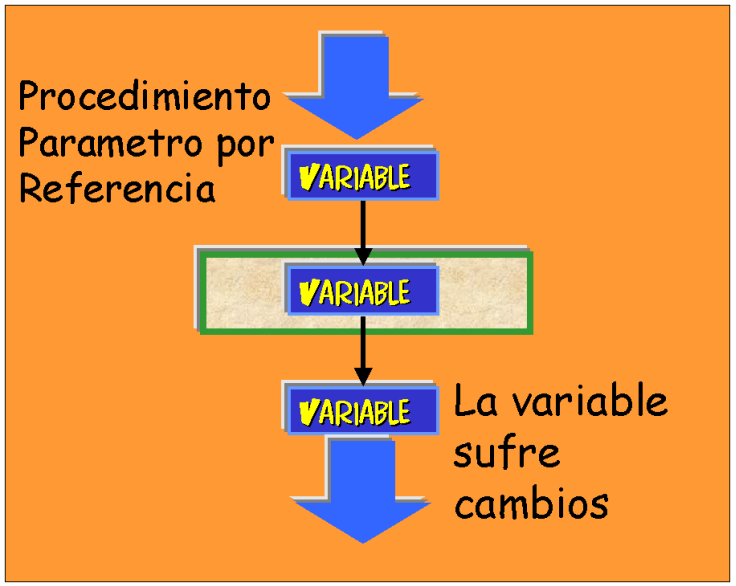
Parámetros

Un parámetro es una variable que utilizamos en la función definition (definición). Es una “handle” (palanca) que permite al código de la función acceder a los argumentos para invocar una función en particular.

```
>>> def saludo(lang):
...     if lang == 'es':
...         print('Hola')
...     elif lang == 'fr':
...         print('Bonjour')
...     else:
...         print('Hello')
...
>>> saludo ('en')
Hello
>>> saludo ('es')
Hola
>>> saludo ('fr')
Bonjour
>>>
```

https://slideplayer.es/amp/17516386/





https://anazan.wordpress.com/2011/10/18/paso-de-parametro-por-valor/

https://anazan.wordpress.com/paso-de-parametros-por-referencia/

Paso por valor y referencia

Dependiendo del tipo de dato que enviemos a la función, podemos diferenciar dos comportamientos:

- **Paso por valor**: Se crea una copia local de la variable dentro de la función.
- **Paso por referencia**: Se maneja directamente la variable, los cambios realizados dentro de la función le afectarán también fuera.

Tradicionalmente:

- **Los tipos simples se pasan por valor**: Enteros, flotantes, cadenas, lógicos...
- **Los tipos compuestos se pasan por referencia**: Listas, diccionarios, conjuntos...

https://docs.hektorprofe.net/python/programacion-de-funciones/paso-por-valor-y-referencia/

```python
def paso_parametros_1(entero):
    entero += 1

def paso_parametros_2(lista):
    lista += [1]

valor = 0
paso_parametros_1(valor)
print(valor)

valor = []
paso_parametros_2(valor)
print(valor)
```

```python
def paso_parametros_1(entero):
    entero += 1

def paso_parametros_2(lista):
    lista += [1]

valor = 0
paso_parametros_1(valor)
print(valor)

valor = []
paso_parametros_2(valor)
print(valor)
```

```
0
[1]
```

TYPE HINTING

What is Type Hints function

- Traditional Python code

```
def twice(num):
    return num * 2
```

- using Type Hints

```
def twice(num: int) -> int:
    return num * 2
```

https://www.slideshare.net/masahitojp/enjoy-type-hints-and-its-benefits-123783453

- If you set wrong Type,We cannot show runtime error

```
def twice(num: int) -> int:
    return num * 2.0

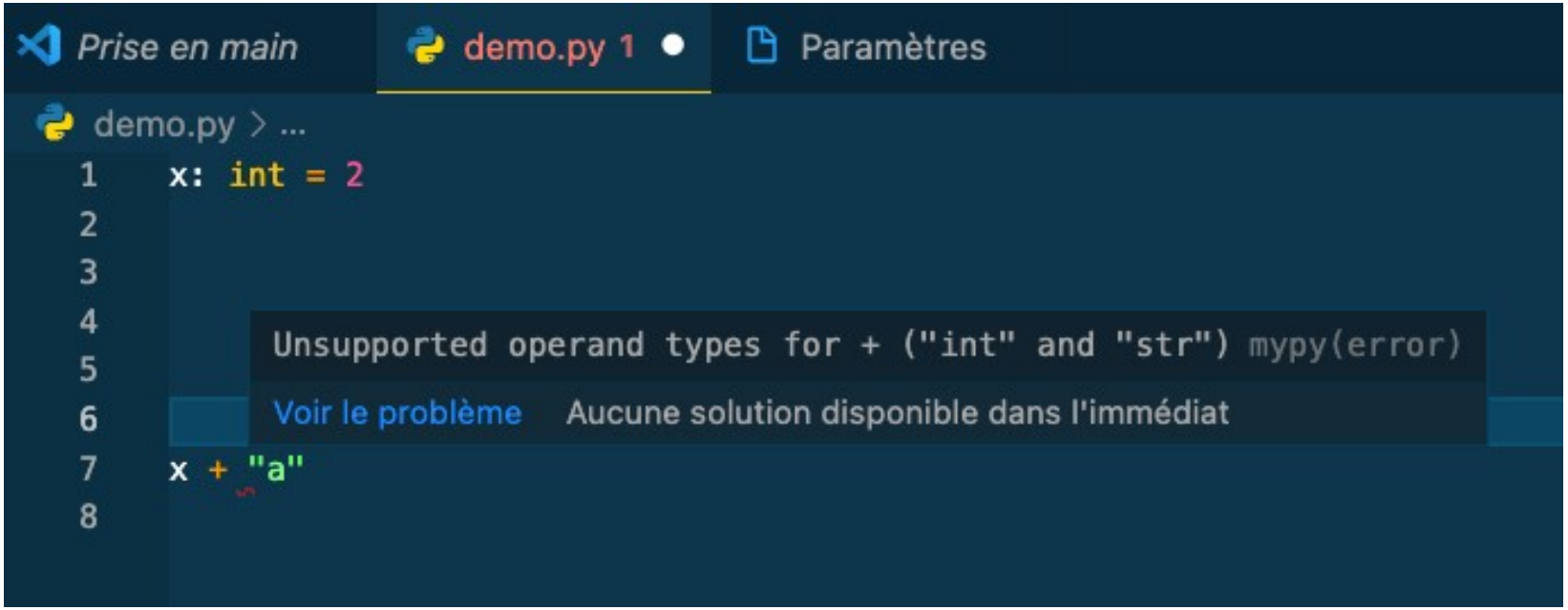
# expected `int` but use `str` 😓
print(twice("hello"))

$ python set_wrong_type.py
$ hellohello ## 😑
```

https://www.slideshare.net/masahitojp/enjoy-type-hints-and-its-benefits-123783453

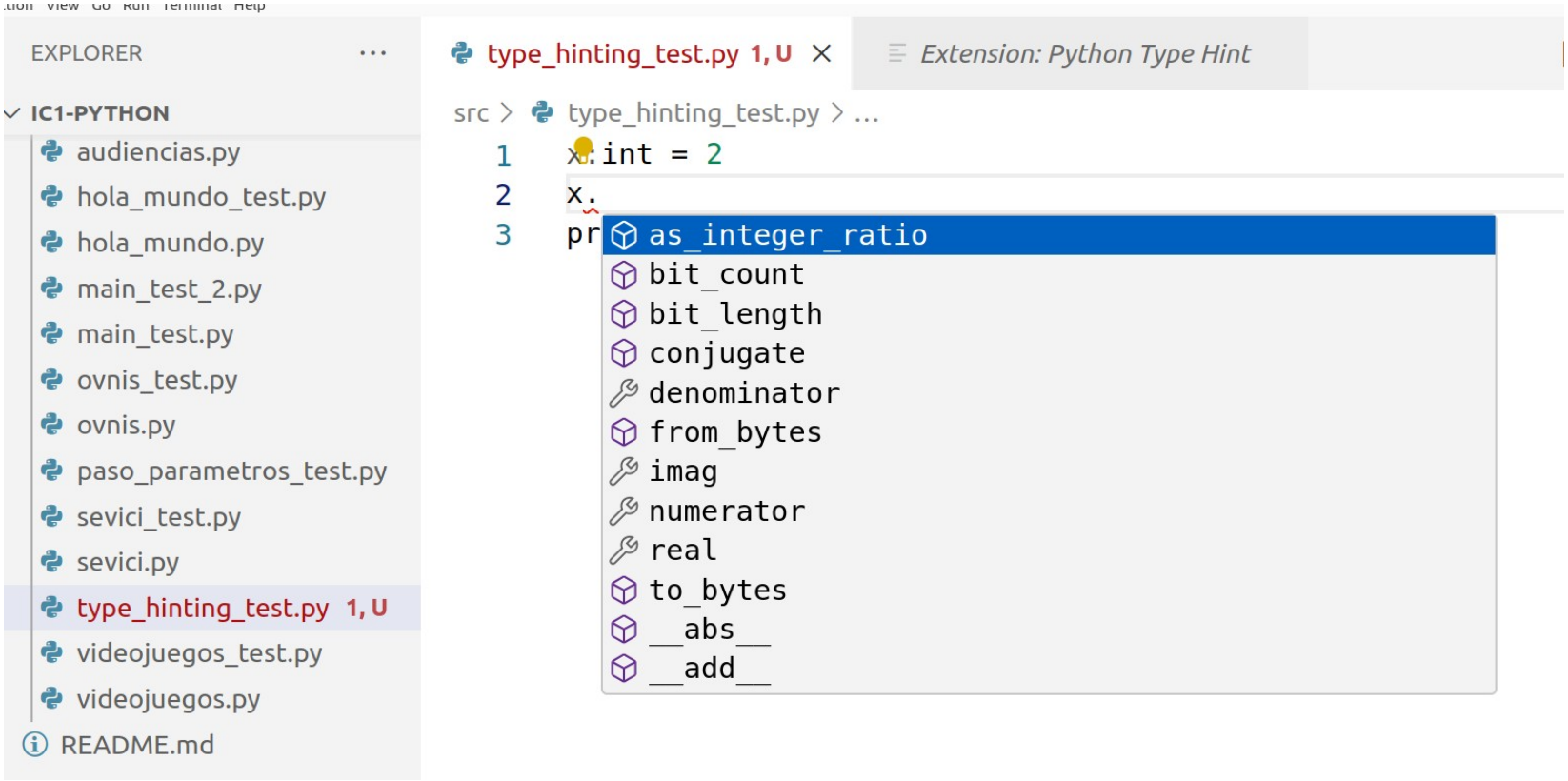
¡¡Python es un lenguaje de tipado dinámico!!

Note: The Python runtime does not enforce function and variable type annotations. They can be used by third party tools such as type checkers, IDEs, linters, etc.



https://docs.python.org/3/library/typing.html

https://towardsdatascience.com/12-beginner-concepts-about-type-hints-to-improve-your-python-code-90f1ba0ac49


EXPLORER
IC1-PYTHON
audiencias.py
hola_mundo_test.py
hola_mundo.py
main_test_2.py
main_test.py
ovnis_test.py
ovnis.py
paso_parametros_test.py
sevici_test.py
sevici.py
type_hinting_test.py 1, U
videojuegos_test.py
videojuegos.py
README.md
type_hinting_test.py 1, U
Extension: Python Type Hint
src > type_hinting_test.py > ...
1 x: int = 2
2 x.
3 pr
as_integer_ratio
bit_count
bit_length
conjugate
denominator
from_bytes
imag
numerator
real
to_bytes
__abs__
__add__

TAREA

- Notebooks de teoría 1, 2 y 3
- Busca e implementa todos los filtros y transformaciones del proyecto SEVICI. Recuerda testear todas tus funciones...